WEEK 01

Responsibility, Evidence, and Relational Thinking

Begin with the system boundary, then rebuild how relations represent facts.

01

The course is organized around operating promises

A database claim is useful only when its scope and evidence are visible.

PROMISE

What should the system protect, permit, return, or recover?

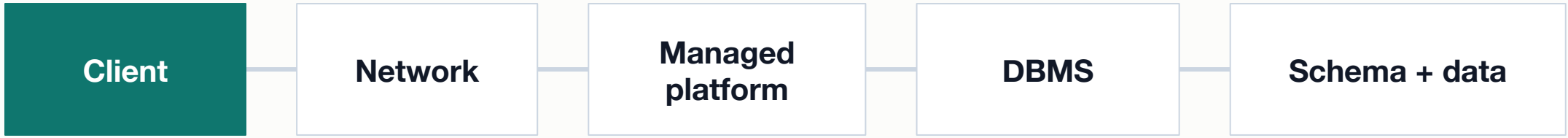
EVIDENCE

What observation would support or challenge that claim?

LIMIT

What remains outside the test or outside your control?

A database request crosses several responsibility layers



A symptom appears at one point; its cause may live at another.

Managed service divides work; it does not erase it

Provider commonly operates

- Physical infrastructure and managed service availability
- Platform patching and supported service components
- Documented controls for networking, identity, and monitoring

Customer still operates

- Data model, constraints, queries, and application behavior
- Database users, roles, policies, and secret handling
- Recovery requirements, verification, and response to mistakes

Evidence turns a symptom into a reproducible observation

A useful record lets another person repeat the observation without guessing.

1. **Environment**
   Service, database, role, tool, and relevant version
2. **Action**
   Exact query, command, or interface operation
3. **Observed**
   Result, error, count, plan, or state
4. **Meaning**
   What the observation supports
5. **Limit**
   What remains untested or uncertain

A screenshot can support evidence but cannot carry it alone

Weak by itself

- Cropped success message with no query
- Dashboard image with no tested identity
- File icon with no restore result

Stronger evidence packet

- Environment and exact action are named
- Observed output includes stable identifiers or counts
- Interpretation and limitation match the test

Lab 1: Map responsibility with evidence

Choose Supabase or Atlas and investigate one control through current official documentation.

- Name one provider responsibility and one customer responsibility.
- Record the official source, environment, and the control you inspected.
- Describe one failure caused by confusing the boundary and one first diagnostic check.

SUBMIT ONE THING / one responsibility-and-evidence record

DAY 2

Rebuild relational thinking before rebuilding SQL

Relations give us a way to reason about facts before syntax hides the question.

A relation represents one kind of fact at a stated grain

Before reading columns, say what one tuple means.

SCHEMA

The relation name, attributes, and intended domains.

INSTANCE

The current tuples stored under that schema.

GRAIN

The real-world fact represented by one tuple.

Keys make identity and relationships testable

One tuple per user

One tuple per ticket

A primary key identifies a tuple; a foreign-key value refers to an existing identity.

Selection keeps tuples; projection keeps attributes

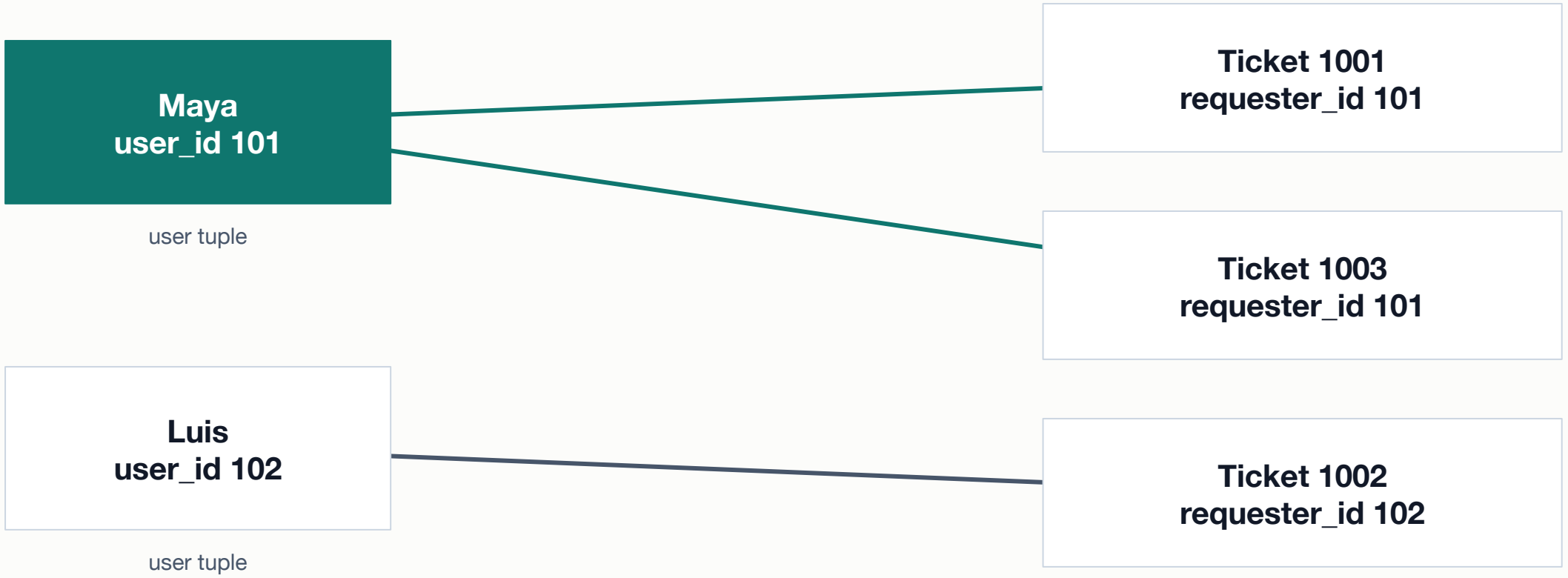
Selection: sigma

- Begins with one relation
- Keeps tuples satisfying a predicate
- Preserves the original attributes

Projection: pi

- Begins with one relation
- Keeps named attributes
- Mathematical projection removes duplicate tuples

A join creates the matching pairs required by a relationship



Maya appears twice after the join because two ticket tuples match her identity.

Relational algebra gives SQL a prediction to test

Move from the question to operations before choosing language syntax.

1. **Question**
   Which open tickets include requester names?
2. **Grain**
   One output tuple per matching ticket
3. **Operations**
   Select open, join requester, project fields
4. **Prediction**
   Expected identifiers, attributes, and count
5. **SQL**
   Execute and compare with the prediction

Lab 2: Represent a question as relational operations

Use the Metro Support relations to reason about rows, attributes, identities, and matching pairs.

- State the grain of users, tickets, and the requested result.
- Mark the selection predicate, projected attributes, and join condition.
- List the expected matching identifiers and one check that could reveal an error.

SUBMIT ONE THING / one relational-reasoning artifact

The first operating habit is to make meaning and evidence explicit

Locate the boundary. State the grain. Predict the result. Record what happened.

- What does one tuple represent?
- Which responsibility layer owns the next check?
- What evidence supports the claim, and what remains untested?